

ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



BLM4531 - BLM4537 Final Raporu

İş Emri Takip Sistemi

Senem Arıcı

22290619

17.01.2026

Github: <https://github.com/senemarici/IsEmriTakipSistemi>

<https://drive.google.com/drive/folders/11TOioU04-yEbceSkvEOym6oT3IVw8bdL?usp=sharing>

ÖZET

Günümüz iş süreçlerinde, gerekli iş operasyonlarının geleneksel yöntemlerle yürütülmesi; Veri tutarsızlığına, iş ortamındaki verimin düşmesine ve geçmişe dönük raporlama eksikliklerine sebep olmaktadır. Bu proje kapsamında, söz konusu sorunlara çözüm üretmek ve süreçleri dijitalleştirmek amacıyla modern yazılım mimarilerine dayalı bir İş Emri Takip Sistemi geliştirilmiştir.

Proje, üç ana modülden oluşan çok katmanlı bir yapı üzerine oluşturulmuştur. Sunucu tarafında, sürdürülebilirlik sağlamak amacıyla .NET 8.0 Core Web API teknolojisi benimsenmiştir. Veritabanı yönetimi için Microsoft SQL Server (MSSQL) tercih edilmiş, veritabanı nesneleri Entity Framework Core ve Code-First yaklaşımı ile kod tabanlı olarak yönetilmiştir.

Kullanıcı arayüzleri tarafında ise hibrit bir yaklaşım takip edilmiştir. Yönetim paneli, kullanıcı deneyimini artırmak amacıyla React.js kütüphanesi ile geliştirilmiştir. Saha personelinin kullanımına sunulması planlanan mobil uygulama ise, Flutter framework'ü kullanılarak tasarlanmıştır. Sistem güvenliği, JWT (JSON Web Token) tabanlı kimlik doğrulama mekanizması ile sağlanmıştır.

Sonuç olarak; yöneticilerin iş ataması yapabildiği, teknisyenlerin sahadan anlık durum bildirebildiği ve tüm veri akışının sekronize bir şekilde yürütüldüğü, uçtan uca çalışan bir sistem sunulması amaçlanmıştır.

İÇİNDEKİLER

ÖZET	i
İÇİNDEKİLER	ii
1. GİRİŞ VE PROJENİN AMACI	1
1.1. Problem Tanımı ve Mevcut Durum	1
1.2. Projenin Amacı	1
1.3. Projenin Hedefleri	1
2.KULLANILAN TEKNOLOJİLER	2
3.YAZILIM MİMARİSİ VE TASARIM	2
3.1. Genel Bakış	3
3.2. Mobil ve Web'in Backend ile İlişkisi	3
4.VERİTABANI TASARIMI	4
4.1. Veritabanı Mimarisi ve Teknoloji Seçimi	4
4.2. Varlık – İlişki Tasarımı	4
4.3. Veritabanı Tabloları ve Görevleri	5
5.BACKEND VE API GELİŞTİRME SÜRECİ	6
5.1. Genel Yaklaşım	6
5.2. Proje Klasör Yapısı ve Bileşenleri	6
5.3. API İletişim Döngüsü	7
5.4. Güvenlik ve Kimlik Doğrulama	7
6.KULLANICI ARAYÜZLERİ GELİŞTİRME SÜRECİ	8
6.1. Web Arayüz (Yönetici Paneli)	8
6.2. Mobil Arayüz (Saha Personeli Uygulaması)	9
7.SONUÇ VE DEĞERLENDİRME	11

1. GİRİŞ VE PROJENİN AMACI

1.1. Problem Tanımı ve Mevcut Durum

Günümüzde özellikle saha operasyonu yürüten teknik servis firmalarında iş takibi hala geleneksel yöntemlerle sürdürülmektedir. Birçok işletmede arıza kayıtlarının telefon görüşmeleri, Excel tabloları veya mesajlaşma uygulamaları üzerinden yönetildiği gözlemlenmiştir.

Bu durum aşağıdaki temel sorunlara yol açmaktadır:

- Veri Kaybı ve Güvenlik Riski,
- İletişim Kopukluğu,
- Performans Ölçülememesi.

1.2. Projenin Amacı

Üzerinde çalıştığım bu projenin temel amacı; yukarıda belirtilen sorunları ortadan kaldıran, web ve mobil platformlarının entegre çalıştığı, güvenli ve ölçeklenebilir bir İş Emri Takip Sistemi geliştirmektir.

Sistem, iki farklı kullanıcı profili üzerine kurgulanmıştır:

1. **Yöneticiler (Web Arayüzü):** İş süreçlerini merkezi bir panelden izleyen, iş emri oluşturan ve personel performansını denetleyen birim.
2. **Saha Personeli / Teknisyenler (Mobil Arayüz):** Kendilerine atanan işleri anlık bildirimle gören, iş detaylarına erişen ve işin durumunu (Başladı/Devam Ediyor/Tamamlandı) güncelleyen birim.

1.3. Projenin Hedefleri

Proje kapsamında gerçekleştirilmesi hedeflenen kazanımlar şunlardır:

Uçtan Uca Dijitalleşme: Arıza bildiriminden işin tamamlanmasına kadar olan tüm sürecin veritabanında kayıt altına alınması.

Gerçek Zamanlı Senkronizasyon: Web panelinde yapılan bir değişikliğin (Örn: Yeni iş ataması), mobil uygulamada anında görüntülenmesi.

Platform Bağımsız Erişim: Sistemin hem masaüstü tarayıcılarda (React) hem de Android/iOS cihazlarda (Flutter) sorunsuz çalışması.

Modern Mimari Kullanımı: RESTful API prensipleri uygulanarak, projenin modüler, test edilebilir ve geliştirmeye açık bir yapıda sunulması.

2.KULLANILAN TEKNOLOJİLER

Projenin geliştirilme sürecinde aşağıdaki teknoloji yığını (Tech Stack) tercih edilmiştir:

- **Backend:** .NET 8.0 Core Web API, Entity Framework Core (Code-First), JWT (Kimlik Doğrulama).
- **Frontend (Web):** React.js, Axios (HTTP İstemcisi).
- **Mobile:** Flutter (Dart Dili).
- **Veritabanı:** Microsoft SQL Server (MSSQL).
- **Araçlar:** Visual Studio, VS Code, Git & GitHub, Swagger (API Dokümantasyonu).

3.YAZILIM MİMARİSİ VE TASARIM

İş Emri Takip Sistemi, istemci-sunucu (Client-Server) prensibine dayalı, merkezi bir RESTful Web API etrafında şekillenen modern bir mimariye sahiptir. Sistem, tek bir çözüm (Solution) içerisinde mantıksal katmanlara ayrılarak geliştirilmiştir.

Backend tarafında, projenin yönetilebilirliğini artırmak ve karmaşıklığı önlemek amacıyla Klasör Bazlı Mantıksal Katmanlama (Logical Layering) tercih edilmiştir. Bu yapıda veritabanı nesneleri, veri transfer nesneleri ve API kontrolcülerini farklı klasörler altında organize edilerek kodun okunabilirliği ve düzeni sağlanmıştır.

Bu mimarinin tercih edilmesinin temel nedeni; iş mantığını tek bir merkezde toplayarak kod tekrarını önlemek, güvenliğini sağlamak ve sistemin gelecekte farklı platformlara kolayca genişletilebilmesine olanak tanımadır.

3.1. Genel Bakış

İş Emri Takip Sistemi, modern yazılım mühendisliği standartlarına uygun olarak İstemci-Sunucu (Client-Server) mimarisi üzerine inşa edilmiştir.

Bu mimaride;

- **Sunucu Tarafı (Backend):** Sistemin beyni olarak görev yapar, veritabanını yönetir ve iş mantığını çalıştırır.
- **İstemci Tarafı (Clients):** Web (React) ve Mobil (Flutter) uygulamaları, sadece veriyi son kullanıcıya sunmakla görevli olan "istemci" rolündedir.

Sistem bileşenleri birbirine sıkı sıkıya bağlı değil, gevşek bağlı (loosely coupled) bir yapıda tasarlanmıştır. Bu sayede, Web arayüzünde yapılan bir güncelleme Mobil uygulamayı etkilememekte; tüm platformlar ortak bir dil (JSON) üzerinden aynı veri havuzunu senkronize biçimde kullanabilmektedir.

3.2. Mobil ve Web'in Backend ile İlişkisi

Projede geliştirilen React (Web) ve Flutter (Mobil) tabanlı istemci uygulamaları, veritabanı ile doğrudan bir bağlantı kurmak yerine, Backend üzerinde çalışan RESTful Web API servislerini tüketerek işlevlerini yerine getirmektedir. İstemci ve sunucu arasındaki bu haberleşme süreci, endüstri standardı olan HTTP/HTTPS protokolleri üzerinden yürütülmekte olup, veri alışverişi platform bağımsızlığını, hafifliği ve insan tarafından okunabilirliği sağlayan JSON (JavaScript Object Notation) formatı ile gerçekleştirilmektedir.

Sistemdeki veri akış döngüsü, kullanıcının Web veya Mobil arayüz üzerinde bir etkileşime girmesiyle (örneğin "Kaydet" butonuna basılması) tetiklenir. Bu aşamada istemci, API'ye işlemin türüne uygun bir HTTP metodu (GET, POST, PUT, DELETE) kullanarak bir İstek yönlendirir. API katmanı gelen bu isteği karşılar, gerekli iş kurallarını uygular, veritabanı işlemlerini tamamlar ve operasyonun sonucunu (Başarılı/Hatalı) bir Cevap olarak istemciye geri döndürür.

4. VERİTABANI TASARIMI

4.1. Veritabanı Mimarisi ve Teknoloji Seçimi

Projenin veri depolama katmanında, veri bütünlüğünü ve tutarlılığını garanti altına almak amacıyla İlişkisel Veritabanı Yönetim Sistemi olan Microsoft SQL Server (MSSQL) tercih edilmiştir.

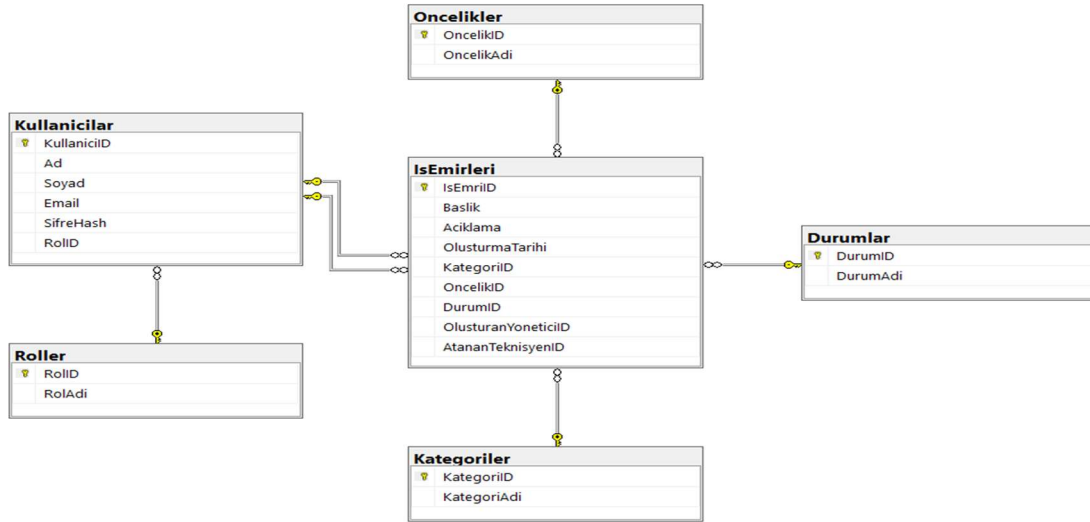
Veritabanı inşasında geleneksel yöntem olan tabloların elle oluşturulması (Database-First) yerine, modern yazılım geliştirme süreçlerinde standart kabul edilen Code-First yaklaşımı benimsenmiştir. Bu yaklaşım sayesinde;

- Veritabanı şeması C# sınıfları (Entities) olarak modellenmiştir.
- Veritabanı üzerinde yapılan değişiklikler Migration dosyaları ile versiyonlanarak kayıt altına alınmıştır.
- Projenin farklı ortamlara taşınması *Update-Database* komutu ile otomatik hale getirilmiştir.

4.2. Varlık – İlişki Tasarımı

Sistem, ilişkisel veritabanı prensiplerine uygun olarak **One-to-Many (Bire-Çok)** ilişkiler üzerine kurulmuştur. Bu tasarım, veri tekrarını önlemek ve normalizasyon kurallarına uymak amacıyla yapılmıştır.

Sistemin temelini oluşturan veritabanı şeması aşağıdaki gibidir:



Şekil 1: Proje Varlık-İlişki (ER) Diyagramı

4.3. Veritabanı Tabloları ve Görevleri

Sistemin veritabanı tasarımı, veri bütünlüğünü sağlamak amacıyla 6 temel tablo üzerine kurgulanmıştır:

- IsEmirleri (Ana Tablo):** Projenin merkezinde yer alır. İşin başlığı, açıklaması ve tarih bilgilerini tutar.
 - Bu tablo Kullanıcılar tablosu ile çift yönlü ilişkiye sahiptir. Hem işi sisteme giren yönetici (OlusturanKullaniciID) hem de işin atandığı teknisyen (AtananPersonelID) ayrı ayrı kayıt altına alınır.
- Kullanıcılar ve Roller:**
 - Kullanıcılar:** Sisteme giriş yapacak personelin kimlik ve şifre bilgilerini saklar.
 - Roller:** Her kullanıcının yetkisini (Yönetici, Teknisyen vb.) belirler ve Kullanıcılar tablosu ile ilişkilidir.
- Tanımlama (Lookup) Tabloları:** Veri tekrarını önlemek ve dinamik bir yapı kurmak için parametrik veriler ayrı tablolarda tutulmuştur:

- **Kategoriler:** Arıza türlerini sınıflandırır (Örn: Donanım, Yazılım).
- **Oncelikler:** İşin aciliyet seviyesini belirtir (Örn: Düşük, Yüksek).
- **Durumlar:** İşin aşamasını takip eder (Örn: Bekliyor, Tamamlandı).

5.BACKEND VE API GELİŞTİRME SÜRECİ

5.1. Genel Yaklaşım

Projenin sunucu tarafı, yüksek performans ve ölçeklenebilirlik gereksinimleri göz önüne alınarak .NET 8.0 Core Web API teknolojisi ile geliştirilmiştir. Mimari tasarımı, uygulamanın yönetilebilirliğini artırmak ve bileşenler arası bağımlılığı azaltmak amacıyla Katmanlı Yapı prensipleri tek bir proje çatısı altında uygulanmıştır.

Backend servisi, dış dünyaya (Web ve Mobil istemcilere) RESTful standartlarına uygun, JSON tabanlı veri döndüren uç noktalar (Endpoints) sunmaktadır.

5.2. Proje Klasör Yapısı ve Bileşenleri

Uygulama mantığı, fiziksel klasörlere bölünmüştür. Her klasörün üstlendiği görev aşağıda detaylandırılmıştır:

- **Models (Varlıklar):** Veritabanı tablolarının kod tarafındaki karşılığı olan "Entity" sınıfları yer alır.
- **Data (Veri Erişim):** Entity Framework Core'un veritabanı ile iletişimini sağlayan ApplicationDbContext sınıfı bu bölümde bulunur. Veritabanı bağlantı ayarları ve tablo konfigürasyonları burada yönetilir.
- **DTOs (Data Transfer Objects):** Veritabanı varlıklarını (Entity) doğrudan dış dünyaya açmak güvenlik riski oluşturabileceğinden, istemcilerle veri alışverişi yapmak için özelleştirilmiş DTO sınıfları (Örn: IsEmriCreateDto, LoginDto) kullanılmıştır.
- **Controllers (Denetleyiciler):** İstemcilerden (Web ve Mobil) gelen HTTP isteklerini (GET, POST, PUT, DELETE) karşılayan sınıflardır. Gelen isteği işler, veritabanı işlemlerini tetikler ve sonucu JSON formatında geri döndürür.

- **Migrations:** Code-First yaklaşımı gereği, model sınıflarında yapılan değişikliklerin veritabanına aktarılmasını sağlayan versiyon dosyaları burada tutulur.

5.3. API İletişim Döngüsü

Bir istemci (Web veya Mobil) sisteme istek gönderdiğinde arka planda şu süreç işler:

1. İstek (Request): İstemci, ilgili Endpoint'e (örn: /api/IsEmirleri) bir istek atar.
2. Yönlendirme (Routing): .NET Core'un Routing mekanizması isteği ilgili Controller'a yönlendirir.
3. İşlem (Action): Controller, gelen veriyi DTO formatında alır, ApplicationDbContext üzerinden veritabanı işlemini yapar.
4. Cevap (Response): İşlem sonucu (Başarılı ise 200 OK, Hata varsa 400 Bad Request) ve talep edilen veri JSON formatında geri döner.

5.4. Güvenlik ve Kimlik Doğrulama

Sistemin güvenlik altyapısında, sunucu kaynaklarını verimli kullanan ve durumsuz (Stateless) bir yapı sunan JWT (JSON Web Token) standardı tercih edilmiştir. Kimlik doğrulama süreci, kullanıcının /api/Auth/Login adresine kimlik bilgilerini içeren bir istek göndermesiyle başlar. Backend servisi, bu bilgileri doğruladığı takdirde kullanıcıya özel, kriptografik olarak imzalanmış bir "Token" (Erişim Anahtarı) üretir. İstemci tarafı, yetki gerektiren sonraki tüm API isteklerinde (örneğin iş emri ekleme) bu token'ı HTTP başlık (Header) bilgisine eklemekle yükümlüdür. Sunucu, gelen her istekte token'ın geçerliliğini kontrol eder ve doğrulama sağlanmadan hiçbir işlemin gerçekleşmesine izin vermeyerek veri güvenliğini garanti altına alır.

6.KULLANICI ARAYÜZLERİ GELİŞTİRME SÜRECİ

Projenin son kullanıcı ile temas ettiği nokta olan arayüz katmanı, farklı kullanım senaryolarına hizmet edecek şekilde iki ayrı platformda kurgulanmıştır. Sistemin yönetim ve raporlama fonksiyonları için geniş ekran avantajı sunan bir Web Paneli, saha operasyonlarının takibi içinse taşınabilirliği esas alan bir Mobil Uygulama geliştirilmiştir. Her iki platform da kendi içinde modern tasarım prensiplerine sadık kalınarak oluşturulmuş olup, kullanıcı deneyimini (UX) en üst seviyede tutmayı hedeflemektedir.

6.1. Web Arayüz (Yönetici Paneli)

Projenin Web tabanlı yönetim modülü, operasyonel verimliliği artırmak ve iş süreçlerini tek bir merkezden yönetebilmek amacıyla geliştirilmiş kapsamlı bir "Kontrol Paneli" (Dashboard) çözümüdür. React.js altyapısı ile inşa edilen bu arayüz, yöneticilere sistemdeki tüm iş akışını kuş bakışı izleme ve anlık müdahale etme yeteneği kazandırmaktadır. Arayüzün temel fonksiyonu, sahadan ve diğer kanallardan gelen ham verileri işleyerek anlamlı bilgi parçalarına dönüştürmek ve karar alma süreçlerini hızlandırmaktır.

Yönetici paneli, sistemdeki veri yoğunluğunu yönetilebilir kılmak adına "Özetleme ve Detaylandırma" prensibiyle çalışmaktadır. Sisteme giriş yapıldığında kullanıcıyı karşılayan özet metrikler; toplam iş yükü, tamamlanan görevler ve aciliyet gerektiren durumlar hakkında sayısal veriler sunarak sistemin anlık durumu raporlamaktadır. Bu özellik sayesinde yöneticiler, detaylar arasında kaybolmadan operasyonel sorunları veya acil müdahale gerektiren noktaları saniyeler içinde tespit edebilmektedir.

Sistemin veri yönetim yetenekleri, gelişmiş bir "Veri Tablosu" yapısı üzerine kurulmuştur. İş emirleri; başlık, kategori, öncelik seviyesi ve atanmış personel gibi kritik parametrelere göre listelenmekte, görselleştirme teknikleri (renk kodları ve etiketler) kullanılarak verilerin okunabilirliği artırılmaktadır. Arayüz, sadece listeleme yapmakla kalmayıp, güçlü bir filtreleme ve arama motorunu da bünyesinde barındırmaktadır. Bu sayede yönetici, yüzlerce kayıt içerisinden belirli bir kategorideki arızaları veya belirli bir personele atanmış işleri filtreleyerek odaklanmış sorgulamalar yapabilmektedir.

Ayrıca Web arayüzü, sistemin Veri Giriş ve Düzenleme merkezi olarak da görev yapmaktadır. Yöneticiler, bu panel üzerinden yeni iş emirleri oluşturarak veritabanına kayıt atabilmekte, hatalı girilen verileri düzenleyebilmekte veya tamamlanmış işleri arşivleyebilmektedir. Tüm bu işlemler, sayfa yenilenmesine gerek duymayan (Asenkron) modern bir kullanıcı deneyimi ile sunulmakta, böylece yönetimsel işlemlerin hızı ve akıcılığı maksimum seviyede tutulmaktadır.

Şekil 2: Yönetici Paneli İş Emirleri Ekranı

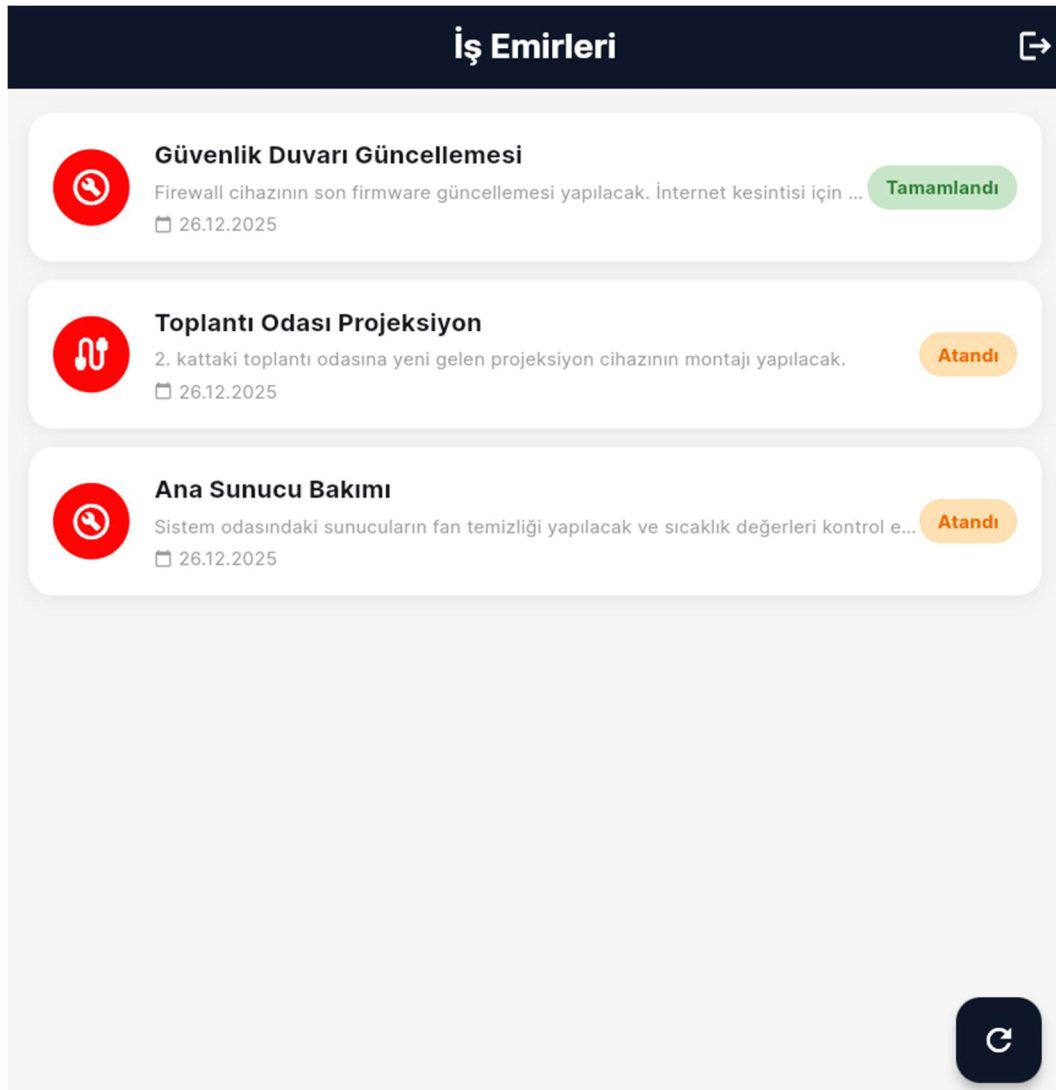
6.2. Mobil Arayüz (Saha Personeli Uygulaması)

Saha personelinin (teknisyenlerin) ofise bağlı kalmadan iş süreçlerini takip edebilmesi amacıyla geliştirilen mobil arayüzde, tek kod tabanıyla hem Android hem de iOS üzerinde çalışabilen Flutter teknolojisi tercih edilmiştir. Tasarım aşamasında karmaşık menü yapılarından kaçınılmış, personelin uygulamaya giriş yaptığında doğrudan iş listesine odaklanabileceği sade ve işlevsel bir arayüz kurgulanmıştır. Görsel tasarımda modern bir kullanıcı deneyimi sağlanmıştır.

Ana ekrandaki iş emirlerinin listelenmesinde Flutter'ın "Card" yapısı kullanılmıştır. Her bir kart üzerinde işin başlığı, açıklaması ve tarih bilgisi yer almaktadır. İşlerin durumunun bir bakışta ayırt edilebilmesi amacıyla renk kodlu etiketler sisteme

entegre edilmiştir. Ayrıca, iş türüne uygun ikonlar kullanılarak listenin okunabilirliği artırılmıştır.

Teknik altyapı olarak uygulama, arka plandaki API ile RESTful mimarisi üzerinden haberleşmektedir. Sahadaki olası veri gecikmelerini veya senkronizasyon ihtiyaçlarını yönetebilmek adına arayüze bir "Yenileme Butonu" konumlandırılmıştır. Kullanıcı bu butonu tetiklediğinde, uygulama API'ye yeni bir GET isteği göndererek listeyi güncellemektedir. Teknisyenin bir iş kartına tıklayıp durumu değiştirmesiyle birlikte veriler veritabanında güncellenmekte ve değişiklikler eş zamanlı olarak yönetim paneline yansımaktadır.



Şekil 3: Mobil Uygulama İş Listesi Ekranı

7.SONUÇ VE DEĞERLENDİRME

Bu proje kapsamında, teknik servis hizmeti veren işletmelerin iş süreçlerini dijitalleştirmek ve verimliliği artırmak amacıyla uçtan uca (Full-Stack) bir İş Emri Takip Sistemi geliştirilmiştir. Proje; veri güvenliğinin, platform bağımsızlığının ve kullanıcı deneyiminin ön planda tutulduğu modern bir yazılım mimarisi üzerine inşa edilmiştir.

Çalışma sonucunda elde edilen temel çıktılar şunlardır:

- **Entegrasyon Başarısı:** Farklı teknolojilerle (C#, JavaScript, Dart) geliştirilen Web ve Mobil uygulamaların, ortak bir dil (JSON) ve merkezi bir API üzerinden sorunsuz bir şekilde haberleştiği kanıtlanmıştır.
- **Mimari Yetkinlik:** Code-First yaklaşımı ve Katmanlı Mimari kullanılarak, projenin sadece çalışan bir koddan ibaret olmadığı, aynı zamanda sürdürülebilir ve geliştirilebilir bir mühendislik ürünü olduğu gösterilmiştir.
- **Veri Güvenliği:** Kullanıcı şifrelerinin hash'lenerek saklanması ve API erişimlerinin JWT (Token) ile korunması sayesinde, temel güvenlik standartları karşılanmıştır.

Bu proje, teorik olarak öğrenilen veritabanı tasarımı, web servisleri ve mobil programlama kavramlarının gerçek hayat senaryosunda birleştirilmesi açısından önemli bir deneyim olmuştur. İlerleyen aşamalarda sisteme; "Anlık Bildirim (Push Notification)" servislerinin eklenmesi gibi geliştirmeler planlanmaktadır.